

# Лабораторна робота №2

## Технології DevOps

### на тему "Continuous Integration"

---

#### Загальні умови

Роботу виконувати на тому самому мікросервісному проєкті, що й лабораторну роботу №1: використовуються ті самі сервіси та вже написані для них Dockerfile. Стек технологій, CI/CD-інструмент та реєстр образів — на власний розсуд; наведені нижче назви інструментів є лише прикладами.

#### 1. Підключити CI/CD до проєкту

- Обрати CI/CD-інструмент на власний розсуд: GitHub Actions, GitLab CI/CD, Jenkins або інший.
- Додати конфігураційний файл пайплайну в репозиторій (.github/workflows/, .gitlab-ci.yml, Jenkinsfile тощо).
- Налаштувати автоматичний запуск пайплайну при кожній зміні у Pull Request (або Merge Request) та при push у робочу гілку.
- Переконатись, що статус виконання пайплайну відображається безпосередньо в інтерфейсі PR.

#### 2. Налаштувати збірку та тестування сервісів

- Для кожного сервісу проєкту передбачити окреме завдання (job) у пайплайні. Однотипні сервіси допускається об'єднати через матрицю (matrix).
- Кожне завдання повинно містити щонайменше два кроки:
  - a) Build — автоматична збірка сервісу (компіляція, packaging, bundling);
  - b) Test — запуск наявних юніт- або інтеграційних тестів.
- Завдання різних сервісів мають виконуватись паралельно та незалежно: падіння одного не повинно скасовувати решту.
- Налаштувати кешування залежностей (npm, pip, go modules, Maven/Gradle тощо) для пришвидшення повторних запусків.

#### 3. Додати статичний аналіз коду

- Підключити лінер або інструмент статичного аналізу відповідно до стека кожного сервісу: ESLint, Ruff, golangci-lint, Checkstyle, SonarQube/SonarCloud, Codacy або інший.
- Інтегрувати аналіз як окремий крок у вже налаштованому пайплайні — до кроку тестування або паралельно з ним.
- Крок має бути блокуючим: виявлені порушення завершують завдання з помилкою.
- Результати перевірки мають бути доступні у Pull Request у вигляді статусу або коментаря.

#### 4. Зібрати контейнерні образи сервісів

- Додати до пайплайну окреме завдання складання образів, яке залежить від успішного завершення всіх завдань збірки та тестування (needs / dependencies / stages).
- Образ будувати для кожного сервісу окремо, використовуючи Dockerfile із лабораторної роботи №1.
- На цьому кроці образ лише збирається локально в межах раннера, без публікації.

- Рекомендовано: додати сканування зібраного образу на вразливості (Trivy, Gype, Docker Scout) як блокуючий крок перед публікацією.

## 5. Опублікувати образи в реєстрі

- Обрати реєстр контейнерних образів: GitHub Container Registry (ghcr.io), GitLab Container Registry, Docker Hub або інший.
- Налаштувати автентифікацію в реєстрі через секрети CI, без збереження облікових даних у репозиторії.
- Публікацію виконувати лише для push у гілки (не для Pull Request).
- Передбачити щонайменше два теги образу:
  - а) унікальний тег збірки (commit SHA або номер запуску);
  - б) тег latest — виключно для основної гілки.
- Продемонструвати опубліковані образи в інтерфейсі реєстру та підтвердити їх працездатність (docker pull + запуск).

## 6. Налаштувати захист основної гілки

- Увімкнути Branch Protection Rules для основної гілки: заборонити прямий push, вимагати Pull Request.
- Позначити всі перевірки пайплайну як обов'язкові (required status checks) — злиття PR має бути заблоковане, якщо будь-який крок завершився з помилкою.
- Продемонструвати заблокований PR: свідомо внести зміну, що ламає тести або лінт, і показати неможливість злиття.

---

## Додаткові матеріали:

1. <https://docs.github.com/en/actions/using-workflows>
2. <https://docs.docker.com/build/ci/github-actions/>
3. <https://github.com/marketplace/actions/build-and-push-docker-images>
4. <https://github.com/marketplace/actions/aqua-security-trivy>
5. <https://github.com/marketplace/actions/official-sonarqube-scan>
6. <https://github.com/marketplace/actions/codacy-analysis-cli>
7. <https://docs.gitlab.com/ee/ci/>
8. <https://www.jenkins.io/doc/book/pipeline/>